CHAPTER 15

DNS & HTTP

Name resolution · resource records · iterative vs recursive · HTTP messages · persistent connections · page-load time

QUICK TRICK

DNS: name → IP, **UDP port 53**, hierarchy **root → TLD → authoritative**. HTTP: **TCP port 80** (443 for HTTPS), stateless. Non-persistent object = **2 RTT**. Persistent + pipelining: base page 2 RTT + **1 RTT** for all objects.

15.1 DNS: the Internet's phone book

● **LEVEL 1 · BEGINNER**

The problem. People use names (www.example.com), routers use IP addresses. **The idea.** A distributed, hierarchical database maps names to addresses. No single server knows everything.

- **Root servers** know who runs each top-level domain (.com, .in ...). **TLD servers** know the authoritative servers of each domain. **Authoritative servers** hold the real records. The **local (resolver) DNS server** asks on the host's behalf and **caches** answers.
- DNS uses **UDP port 53** (TCP for zone transfers or responses that do not fit in a UDP message).

Record type	Maps	Example
A	Hostname → IPv4 address	www.x.com → 93.184.216.34
AAAA	Hostname → IPv6 address	
NS	Domain → name of its authoritative server	x.com → ns1.x.com
CNAME	Alias → canonical name	www.x.com → host1.x.com
MX	Domain → mail server name	x.com → mail.x.com
PTR	IP → name (reverse lookup)	

A record is (Name, Value, Type, TTL). The **TTL** says how long a cache may keep it.

15.2 How a lookup runs

● **LEVEL 2 · EXAM**

```
Host --1--> Local DNS --2--> Root          "ask .com TLD"      (iterative: referral)
                      <-3--
                      --4--> TLD (.com)    "ask ns1.x.com"
                      <-5--
                      --6--> Authoritative "www.x.com = IP"
                      <-7--
Host <--8-- Local DNS      (host asked recursively, local DNS asked iteratively)
```

- **Recursive query:** "get the whole answer for me". **Iterative query:** "tell me who to ask next" (a referral).
- Empty caches: the local server contacts root, TLD and authoritative = **3 external queries** plus the host's own query = 4 query/response pairs = **8 messages** (also 8 if all are recursive).
- Time = RTT(host → local) + RTT(local → root) + RTT(local → TLD) + RTT(local → authoritative). With the TLD or authoritative server cached, steps drop out.

15.3 HTTP: the web's language

● **LEVEL 1 · BEGINNER**

HTTP is an **application-layer** request/response protocol over **TCP**. It is **stateless**: the server keeps no memory of earlier requests (cookies add state).

AMJESH (https://singhamjesh.github.io/)

```
Request                                  Response
GET /index.html HTTP/1.1                 HTTP/1.1 200 OK
Host: www.example.com                    Content-Type: text/html
Connection: keep-alive                   Content-Length: 1250
(blank line, then optional body)         (blank line)  <html> ... </html>
```

Methods	Status codes
GET (fetch), POST (send data), HEAD (headers only), PUT, DELETE	200 OK · 301 Moved Permanently · 304 Not Modified · 400 Bad Request · 404 Not Found · 500 Internal Server Error

- **Cookies:** server sends Set-Cookie; the browser returns it later. **Conditional GET** (If-Modified-Since) → 304 saves bandwidth. **Web caches / proxies** cut delay and traffic.
- **HTTP/1.0** default is non-persistent. **HTTP/1.1** default is **persistent** (keep-alive) and allows **pipelining**.

15.4 Page-load time: the RTT counting game

● LEVEL 3 · GATE

Page = 1 base HTML file + n small objects. Ignore transmission time unless given. **One TCP setup = 1 RTT, one request/response = 1 RTT.**

Mode	Time	n = 10, RTT = 100 ms
Non-persistent, serial	(n + 1) × 2 RTT	11 × 200 = **2200 ms**
Non-persistent, m parallel connections	2 RTT + ⌈n/m⌉ × 2 RTT	m = 5: 200 + 2 × 200 = **600 ms**
Persistent, no pipelining	2 RTT + n × RTT	200 + 1000 = **1200 ms**
Persistent, with pipelining	2 RTT + 1 RTT	**300 ms**

If transmission times are given, add them for every object. If a DNS lookup is needed first, **add the total DNS time at the start** (RTT_1 + RTT_2 + ... for each DNS server contacted).

15.5 Common mistakes

COMMON MISTAKE

- Forgetting the extra RTT of the TCP handshake for non-persistent connections.
- Adding the DNS time once **per object** (it is cached after the first lookup).
- Saying HTTP is stateful (it is stateless).
- Mixing up recursive (server does all the work) and iterative (server returns a referral).

QUICK TRICK

Memory line: "DNS first, then handshake, then request. Persistent saves the handshake; pipelining saves the waiting."

GATE QUICK CHECK — Chapter 15

```
DNS: UDP 53 | A, NS, CNAME, MX, PTR | root -> TLD -> authoritative | cache with TTL
HTTP: TCP 80/443 | stateless | GET/POST/HEAD | 200 301 304 404 500
non-persistent serial (n+1)*2RTT | parallel 2RTT+ceil(n/m)*2RTT
persistent 2RTT + n*RTT | pipelined 2RTT + RTT | add DNS time once at start
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Transport protocol and port for DNS queries?

Q2. Which record maps a hostname to an IPv4 address? an alias to a name? a domain to its mail server?

Q3. Default port for HTTP and HTTPS?

Q4. Is HTTP stateful or stateless? How is state kept?

● LEVEL 2 · EXAM

Q5. Page with 1 base file + 5 objects, RTT = 50 ms, non-persistent serial. Time (ignore transmission)?

Q6. Same page, persistent without pipelining.

Q7. Same page, persistent with pipelining.

Q8. Status codes for "Moved Permanently", "Not Modified", "Not Found".

● **LEVEL 3 · GATE**

Q9. A browser resolves a name via three DNS servers with RTTs 10, 20 and 30 ms, then fetches a base file and 4 objects (non-persistent, serial, HTTP RTT = 50 ms). Total time?

Q10. Same, but the 4 objects use 4 parallel non-persistent connections.

Q11. Messages exchanged to resolve a name with all caches empty (local server uses iterative queries).

Q12. The local DNS server already caches the TLD server for this domain. How many query/response pairs now?

Answers

Q	Solution
1	**UDP port 53**.
2	**A**, **CNAME**, **MX**.
3	**80** and **443**.
4	**Stateless**; **cookies** add state.
5	6 × 2 × 50 = **600 ms**.
6	2 × 50 + 5 × 50 = **350 ms**.
7	3 × 50 = **150 ms**.
8	**301, 304, 404**.
9	DNS = 60 ms. HTTP = 5 × 2 × 50 = 500 ms. Total **560 ms**.
10	60 + 100 (base) + 100 (all 4 in parallel) = **260 ms**.
11	4 queries + 4 responses = **8 messages**.
12	host → local, local → TLD, local → authoritative = **3 pairs** (root skipped).

Next: Chapter 16 — Out-of-Syllabus Quick Sheets (ARP, DHCP, ICMP, SMTP/FTP, framing, bridging, OSI).